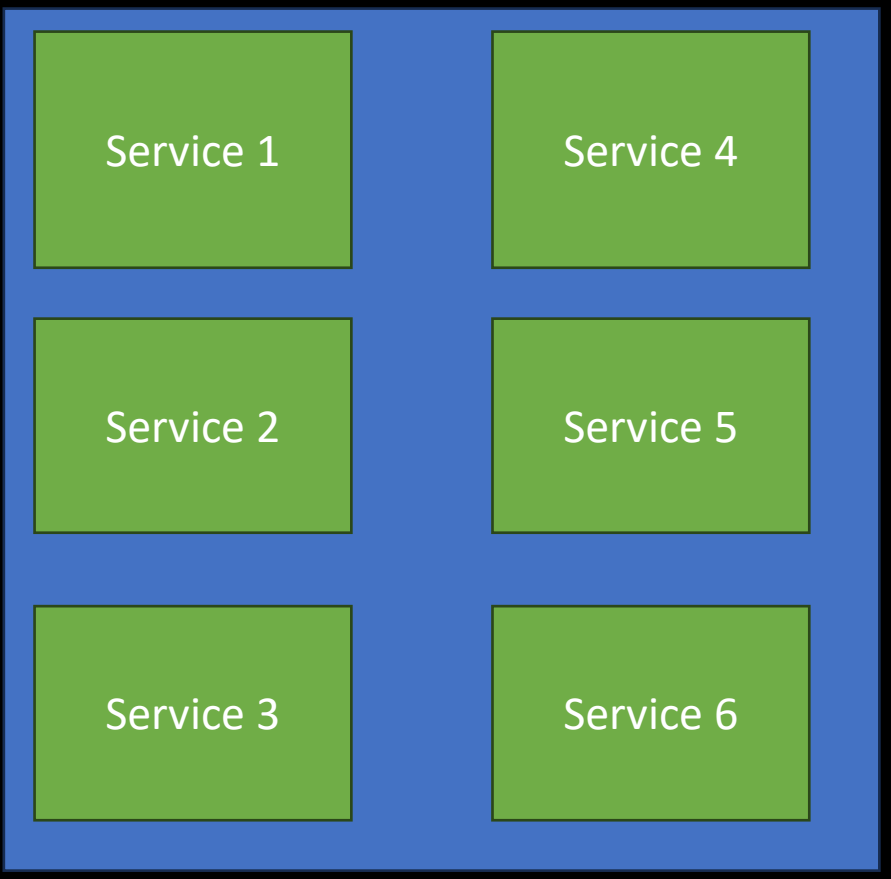
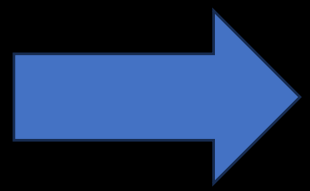
Micro services

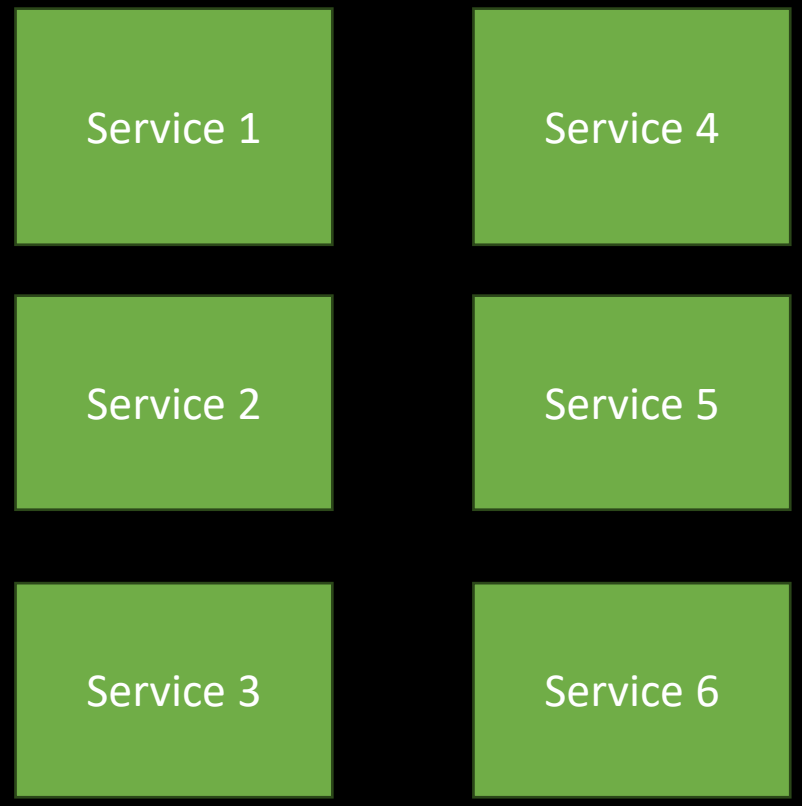


- ✓ What is Microservice ?
- ✓ Examples of Microservice
- ✓ Communication between Microservice
- ✓ CI / CD for microservices
- ✓ Benefits / Challenges of MS

What is micro service ?







Break a big application into smaller ones (microservices)



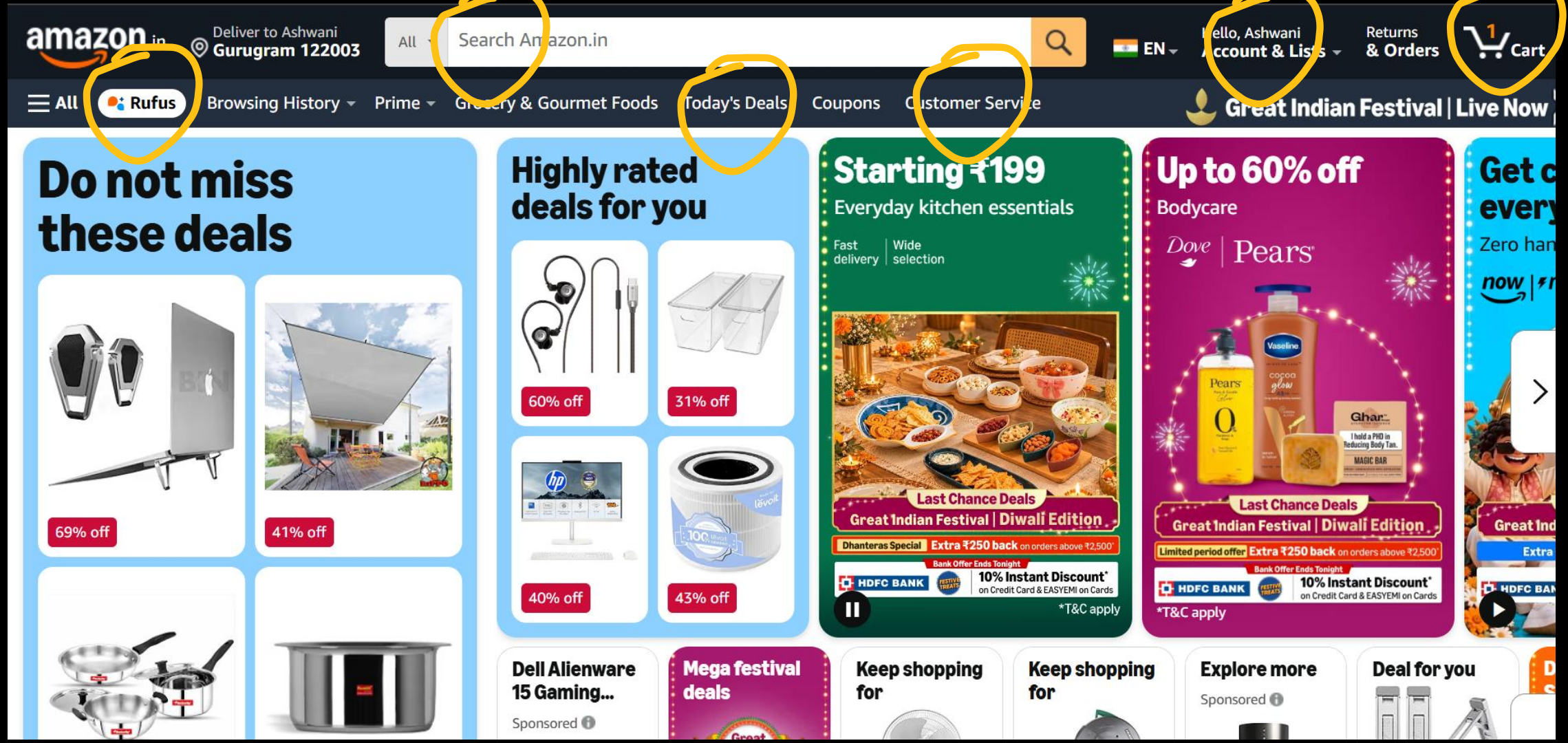
Example:

An ecommerce website might have separate services for:

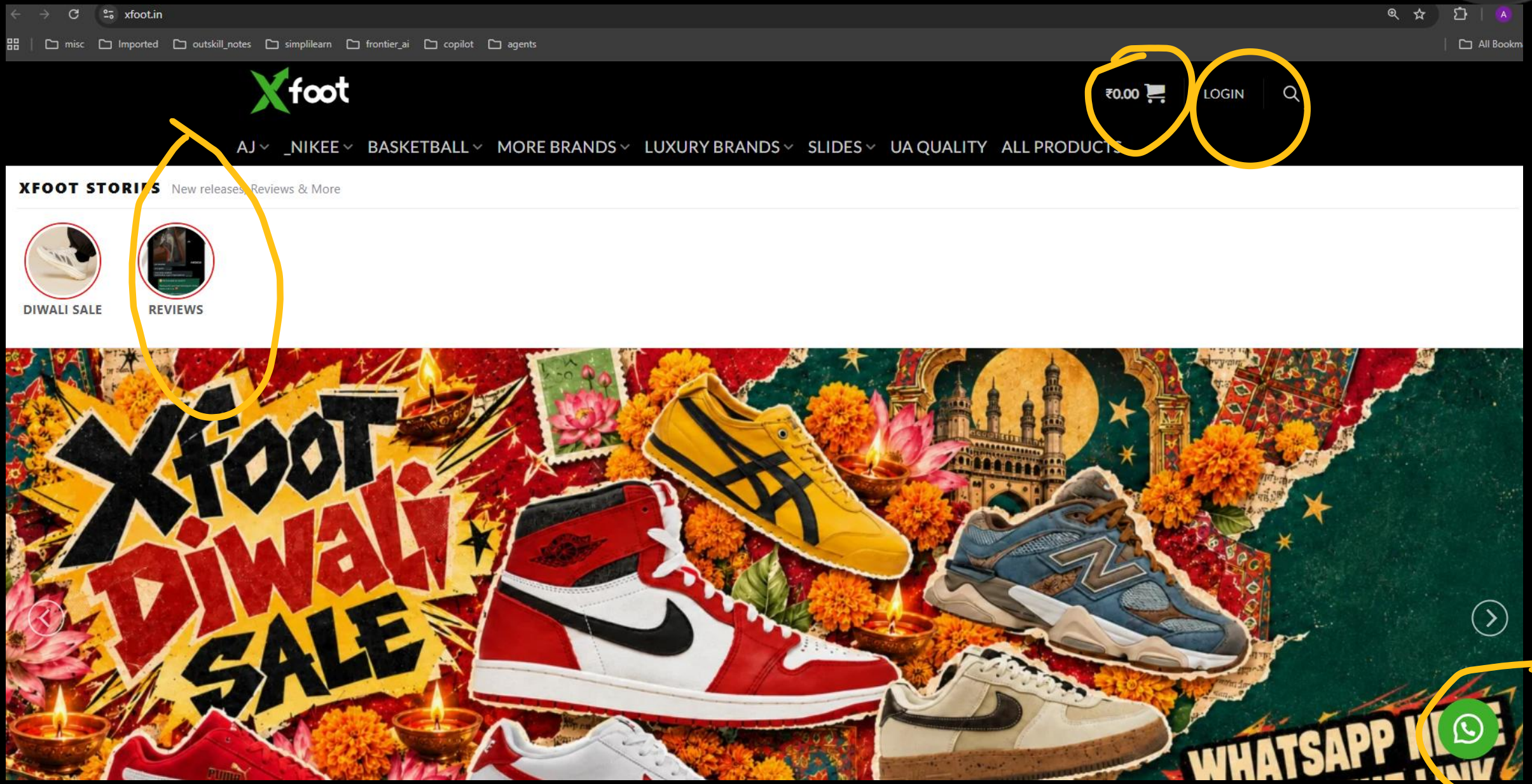
- ✓ Managing products – Buy / sell items
- ✓ Maintaining shopping carts – Add / remove / update items
- ✓ Handling payments
- ✓ Tracking inventory
- ✓ Providing AI recommendations

Together, these services power one shopping experience.

Example of microservice

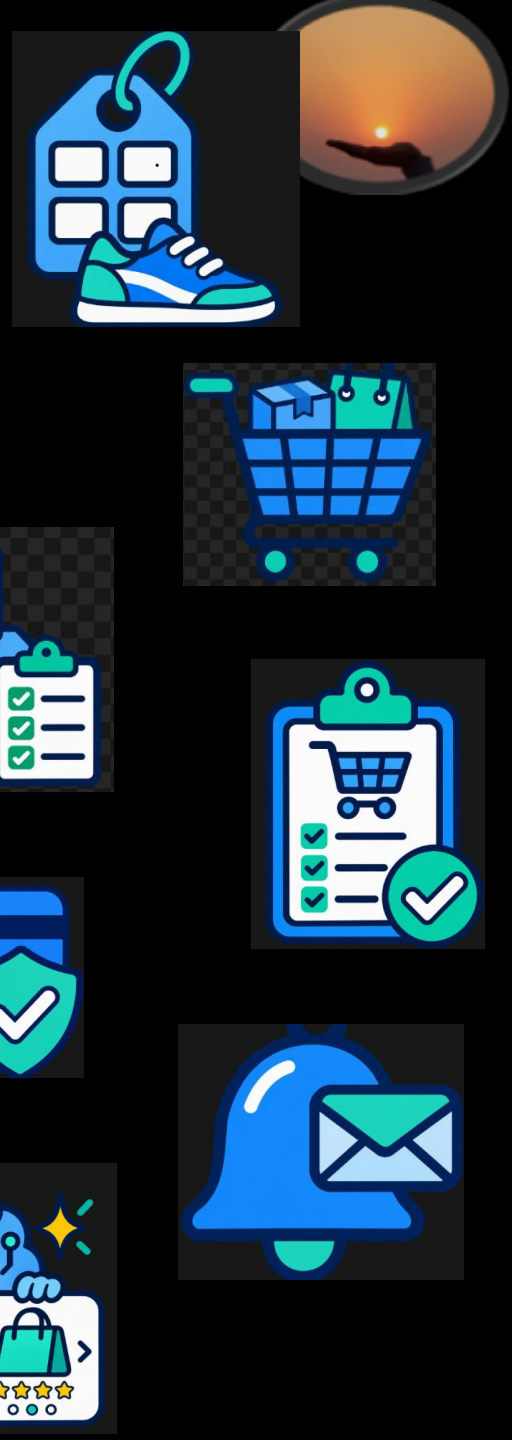


Example of microservice



Each service owns its business logic and data. It is self-contained + independant

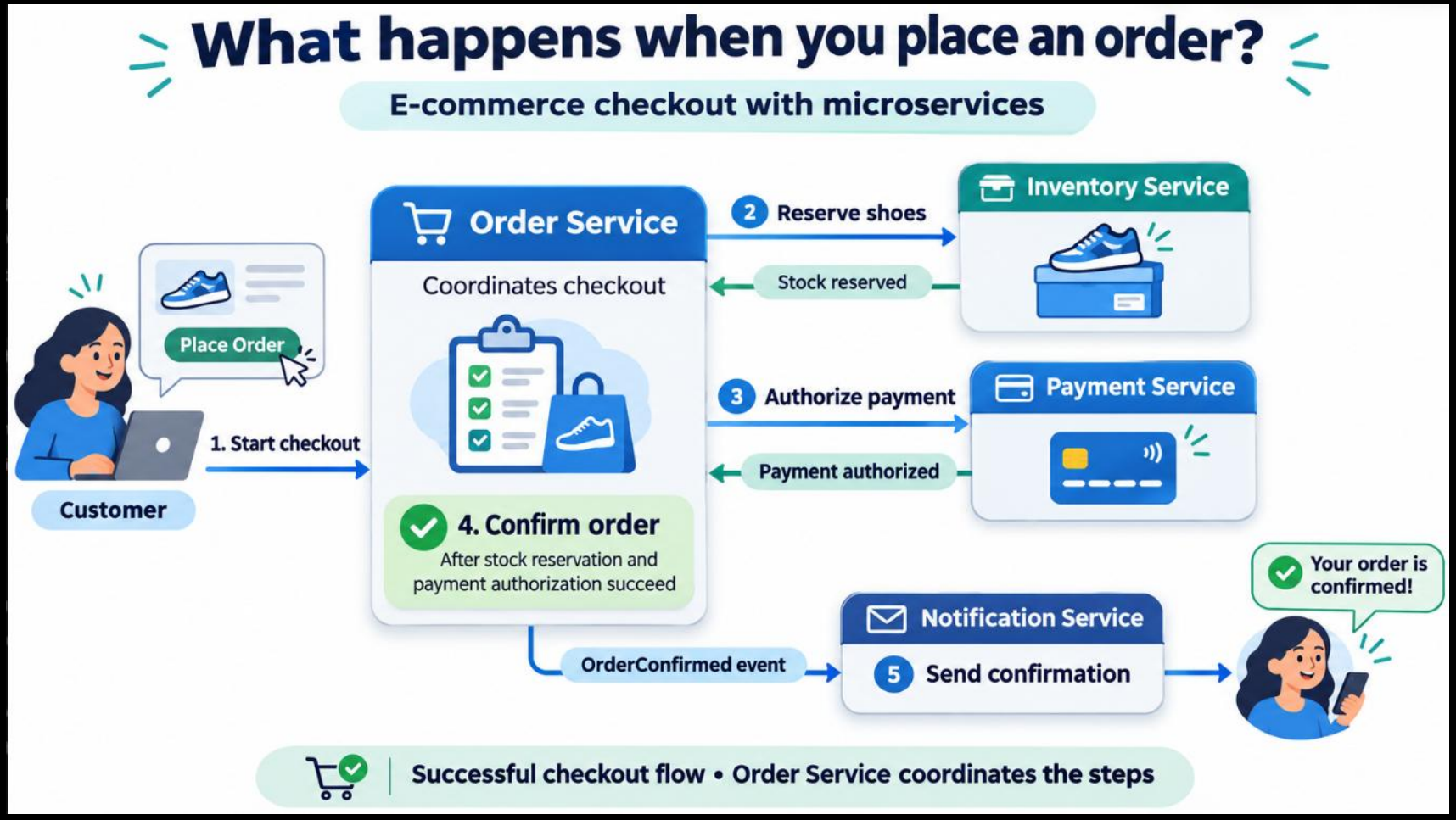
Service	Main responsibility
Product Service	Stores product descriptions, prices, and images
Cart Service	Tracks the products a customer wants to buy
Inventory Service	Tracks available stock and reserves items
Order Service	Creates orders and tracks their status
Payment Service	Coordinates payments with a payment provider
Notification Service	Sends order confirmations and delivery updates
AI Recommendation Service	Suggests products relevant to the customer



Example - Checkout

Imagine an ecommerce website called **xFoot.in**. When the customer clicks **“Place Order”**:

1. The **Order Service** starts the checkout process.
2. The **Inventory Service** attempts to reserve the shoes.
3. The **Payment Service** requests payment authorization.
4. The **Order Service** confirms the order when the required steps succeed.
5. The **Notification Service** sends a confirmation.

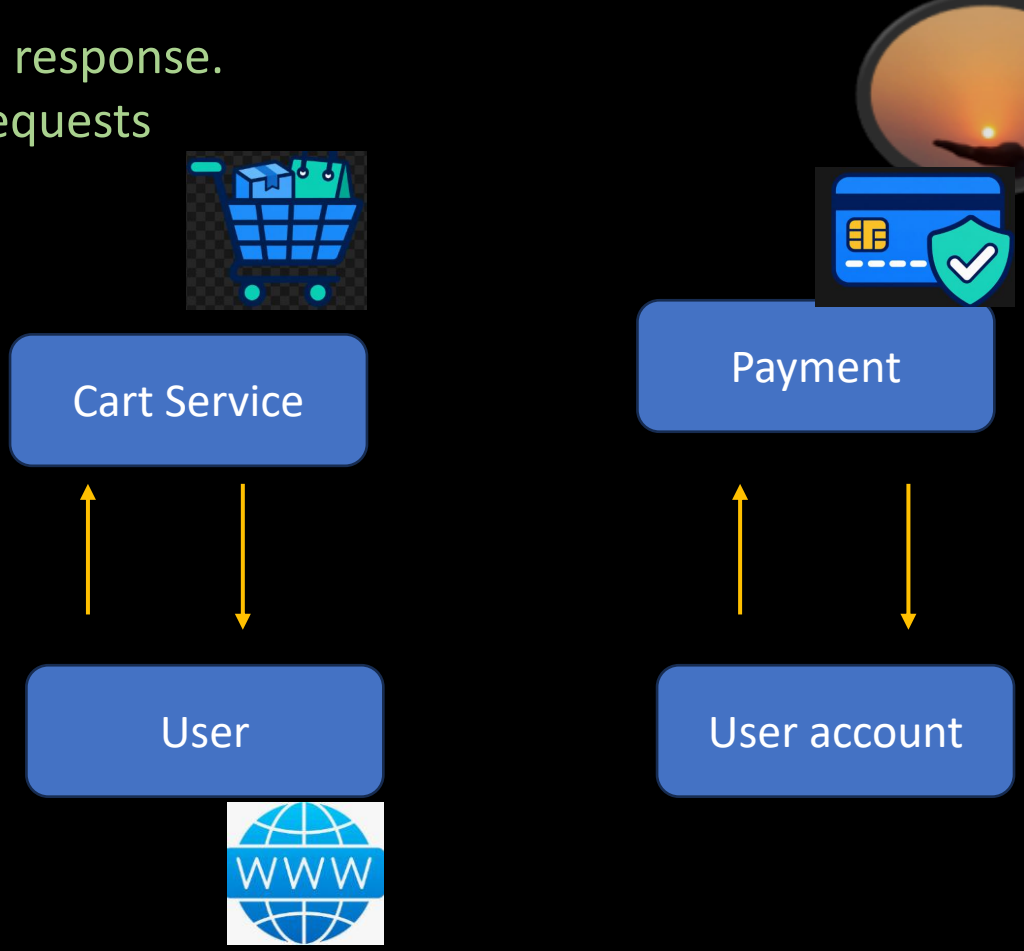




Communication between services

2 main ways: **Synchronous** requests and **Asynchronous** messages.

Synchronous : One service calls another service and ***waits*** for a response.
Services can communicate with each other through HTTP API requests



Adding a Product to the Cart

1. The User clicks **"Add to Cart "** on website. It sends an API request.

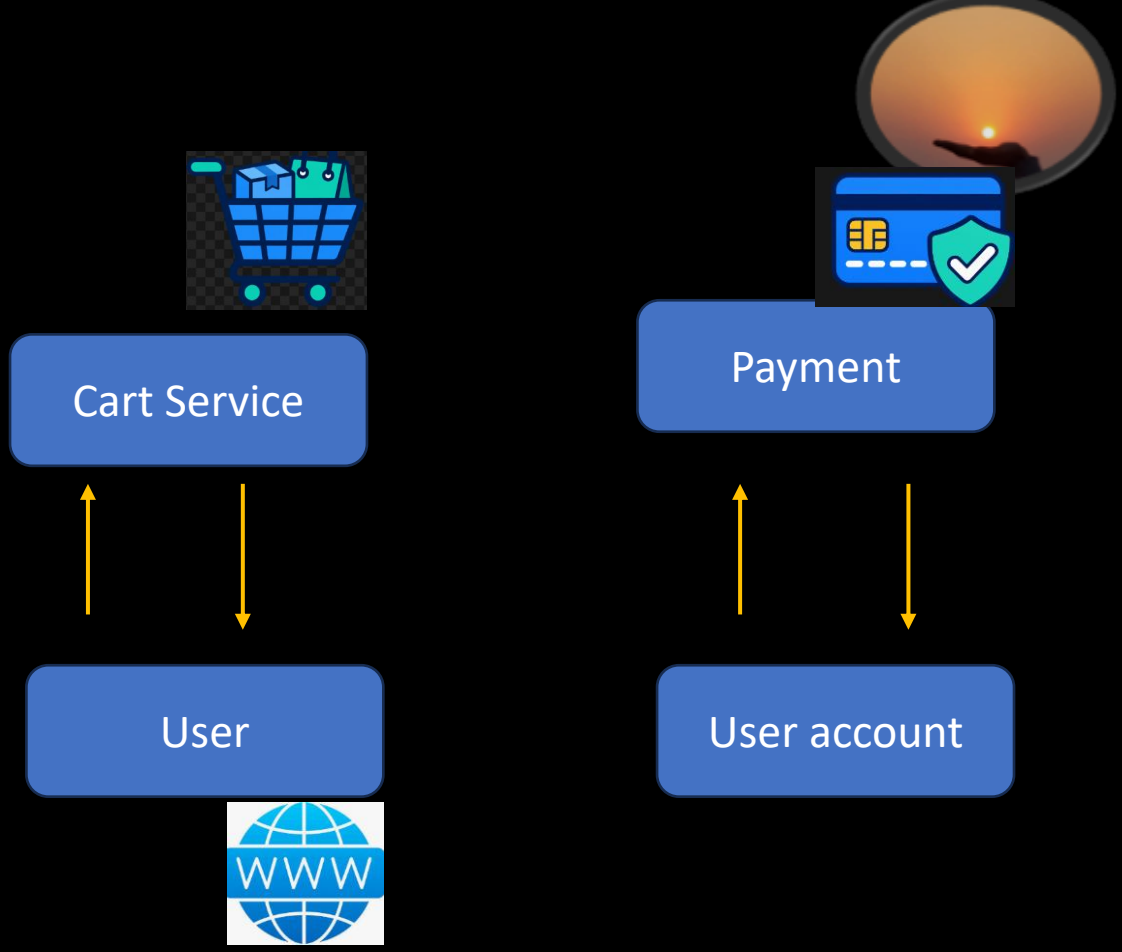
Example API request: POST /cart/items

- *Information sent: Product ID: SHOE-101; Quantity: 1*

2. The Cart Service records the product and quantity.
3. The service returns the updated cart.

Information returned: Updated cart contents

4. The website displays the result top the user.

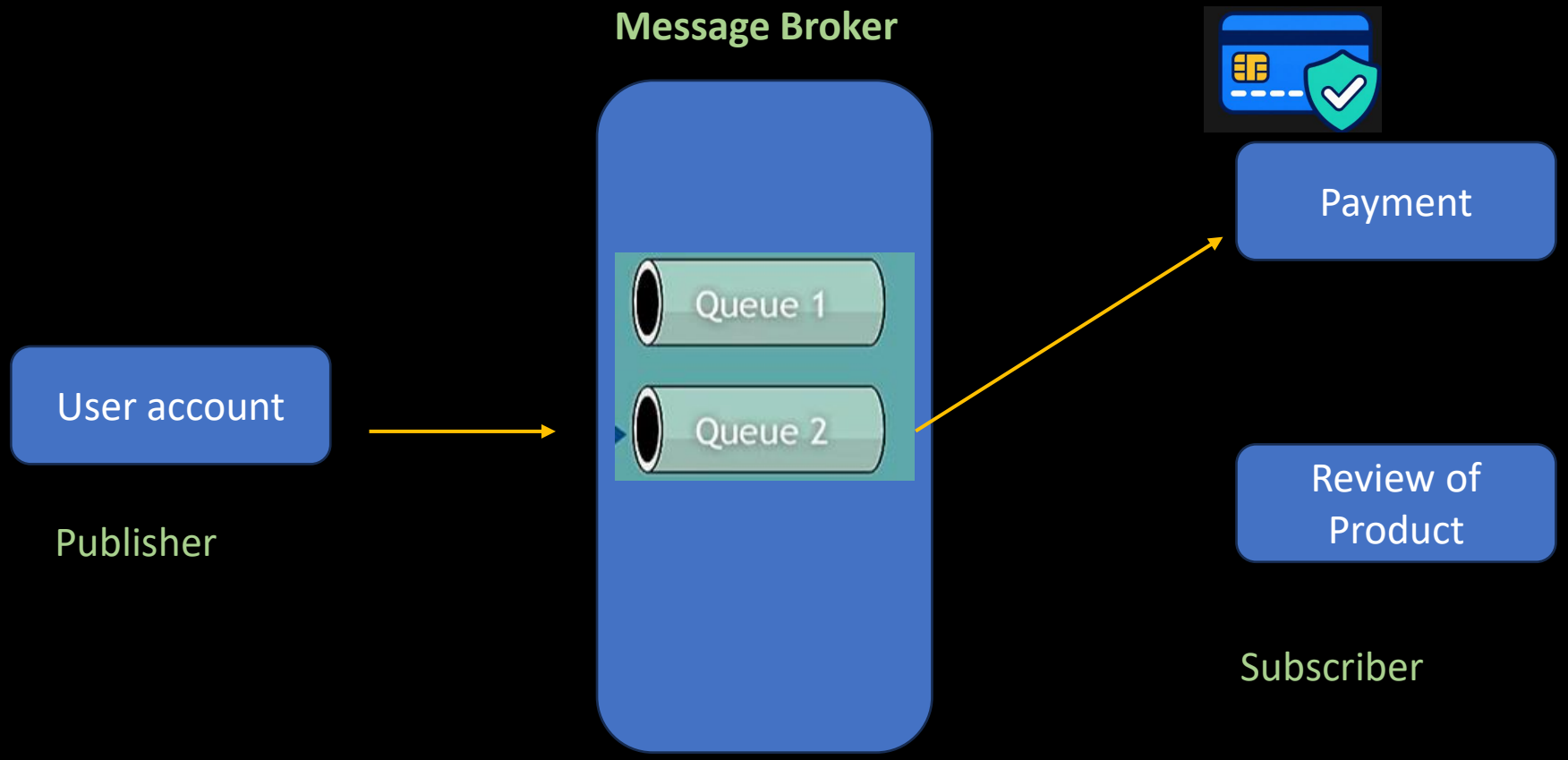


Main challenge of synchronous:
If the other service is slow or unavailable, the caller is affected.
Use timeouts and handle failures.

Asynchronous:

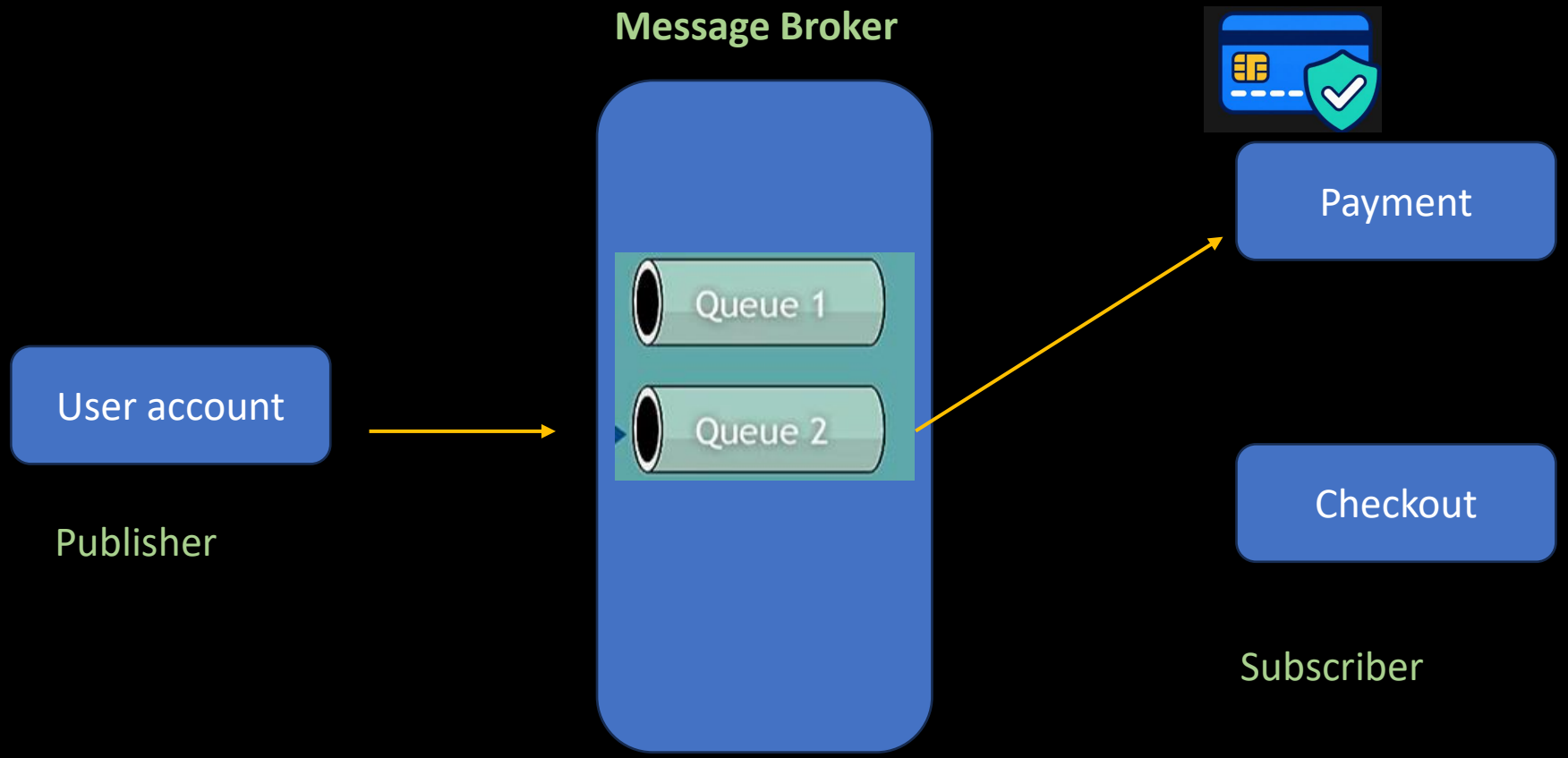
A service sends a message through a **message broker**. Another service processes it later.

The sender does ***not wait*** for the business task to finish.



Example: RabbitMQ, Kafka, and cloud messaging services

Asynchronous:
Use when: work takes time, traffic arrives in bursts, or several services should react independently.



Main challenge:
Results arrive later, and messages may be delivered more than once. **Solution:** Services need retry handling and protection against duplicate processing.

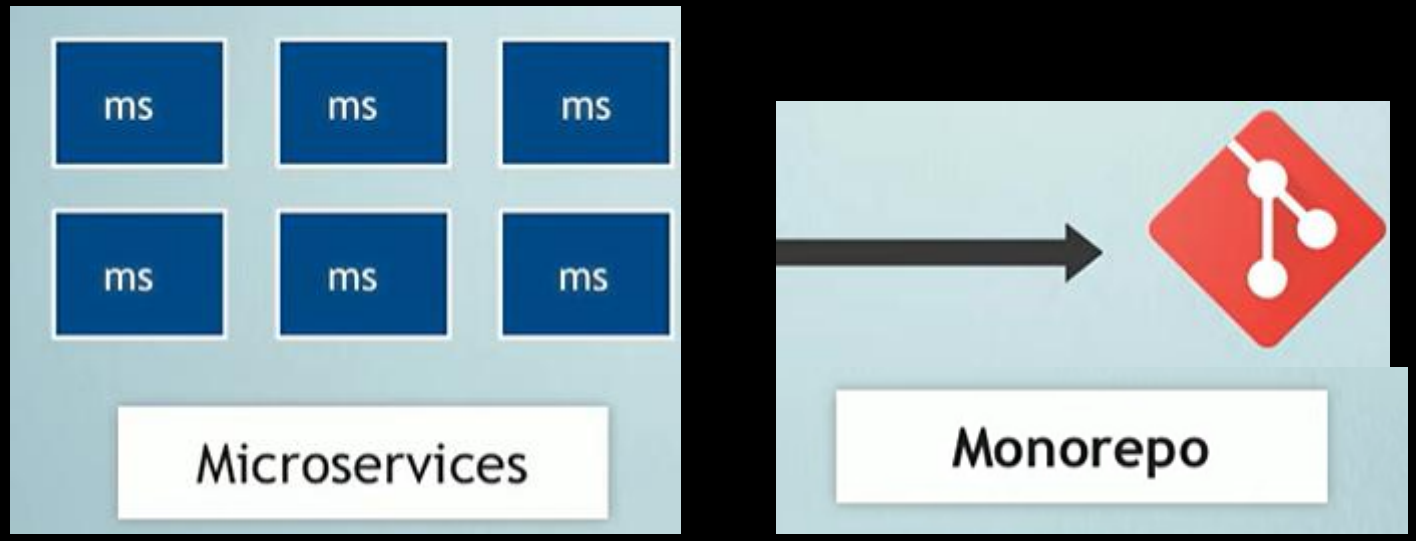


CI / CD Pipeline for microservices

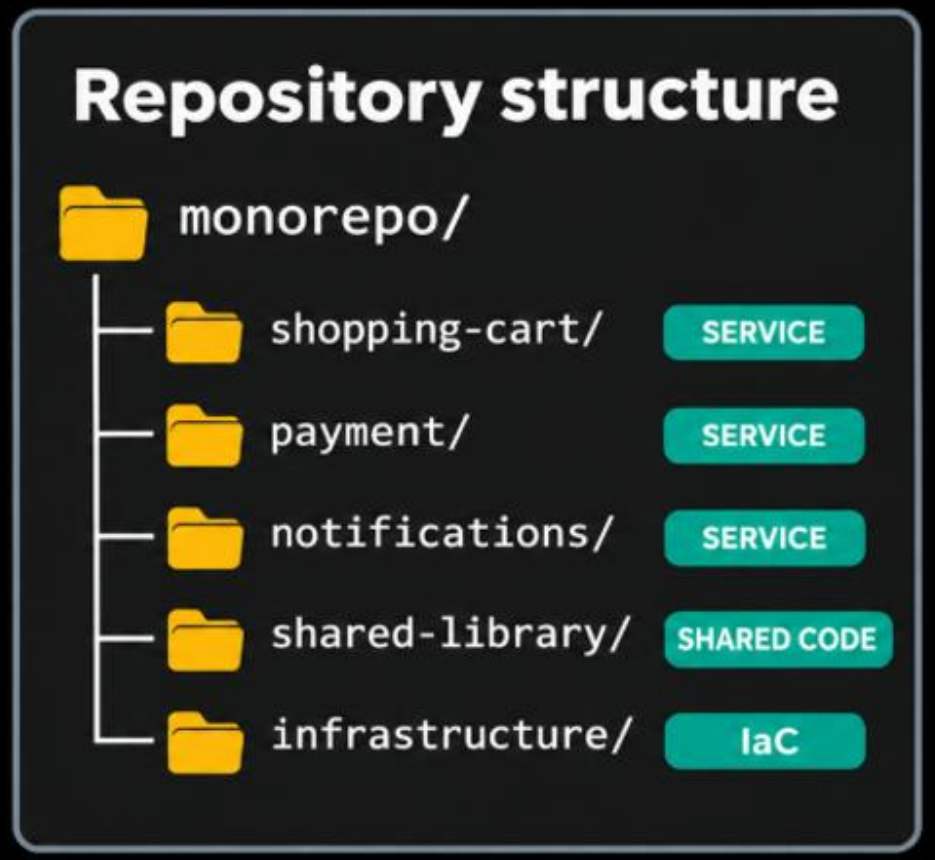
1. Mono Repo
2. Poly Repo

Mono Repo

Single repo contains many projects/microservices



Mono Repo



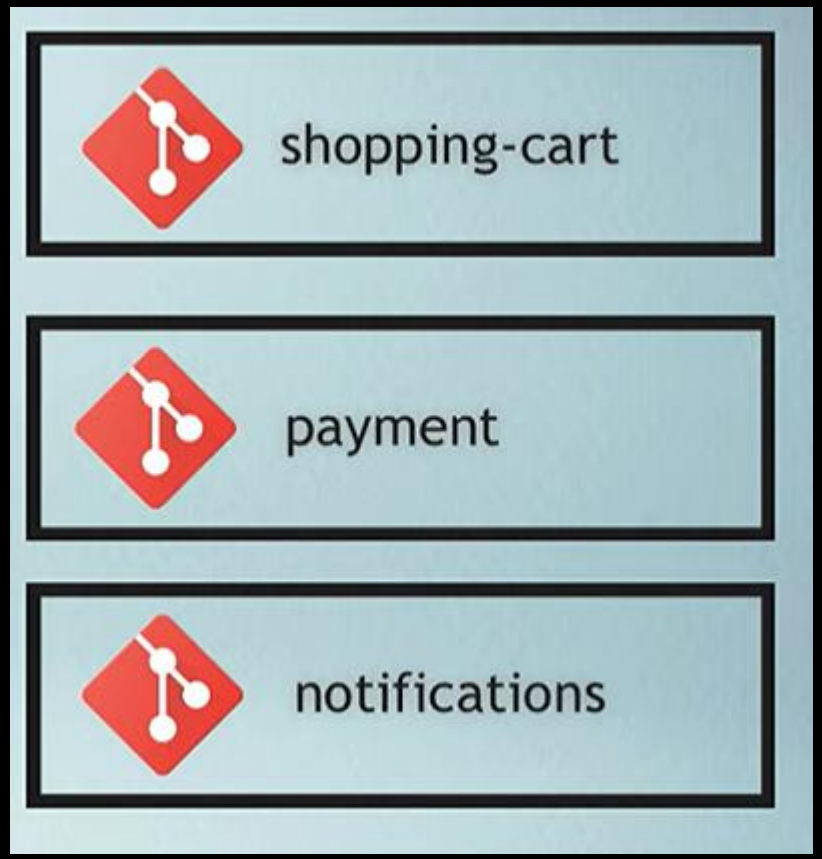


Mono Repo good for

- small projects.
- when you want one CI/CD pipeline.

Poly Repo:
1 repo for each service

Code is isolated -> Developers can clone and work on them separatly



When to Use:

- Many services and each using different frameworks (Java, Python, etc.)
- You have separate teams for each service. Each team will have separate CI/CD pipeline
- You want your code base to be small

Benefits for Microservices



Each microservice team can choose its own tech stack

User Account



Shopping cart



Payment



Each team can work independently

Update 1 microservice without redeploying others.

Recommendation



Contain some failures: **Recommendations service** can be unavailable while **Shopping service** continues.

Challenges of MS



- More services to deploy, monitor, and secure.
- Network calls can be slow or fail.



- More complexity: Keeping orders, payments, and inventory consistent is harder.

Demo of Microservice

